Лекция 6: Порождающие паттерны GoF. Factory Method, Abstract Factory, Builder. Анализ антипаттерна Singleton

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Order → Emergence (От принципов SOLID к проверенным паттернам проектирования)

Введение и инженерный контекст

На первой половине курса мы выстроили фундаментальную базу: инкапсуляцию, композицию, полиморфизм и 5 принципов SOLID.
Теперь мы переходим к каталогу **шаблонов проектирования GoF** (*Gang of Four* — Эрих Гамма, Ричард Хелм, Ральф Джонсон, Джон Влиссидес), опубликованному в 1994 году.
Паттерны GoF — это не готовые библиотеки или алгоритмы, а **проверенные временем архитектурные рецепты решения типовых задач проектирования**.
Паттерны делятся на три семейства: 1. **Порождающие (Creational)** — абстрагируют процесс создания объектов; 2. **Структурные (Structural)** — организуют классы и объекты в крупные устойчивые структуры; 3. **Поведенческие (Behavioral)** — организуют алгоритмы взаимодействия и распределение обязанностей.
Сегодня мы детально изучим **порождающие паттерны**: Factory Method, Abstract Factory, Builder, а также проведем критический инженерный анализ самого известного и опасного паттерна — **Singleton (Одиночка)**.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Проблема прямого инстанциирования (new / ClassName()). Зачем нужны порождающие паттерны	Архитектурная проблема
2	00:20–00:45	Паттерны Фабричный метод (Factory Method) и Абстрактная фабрика (Abstract Factory)	Теория + код

№	Время	Тема	Формат
3	00:45–01:10	Паттерн Строитель (Builder): пошаговое конструирование сложных отчетов и конфигураций	Практика
4	01:10–01:25	Singleton: почему это антипаттерн в современной разработке и как управлять жизненным циклом через DI	Анализ ловушек
5	01:25–01:30	Подведение итогов и связь с лабораторной работой №6	Рефлексия

---

1. В чем проблема прямого вызова ClassName()?

Когда в бизнес-логике пишется прямой вызов конструктора:

```
def process_ticket_purchase(self):
    ticket = PaperQrTicket(fare=100.0, bus_id="02-123") # Прямое зацепление за конкретный класс!
```

Код бизнес-логики намертво привязывается к конкретной реализации PaperQrTicket. Если завтра потребуется выпускать виртуальные билеты или Bluetooth-токены, придется модифицировать весь клиентский код.

Порождающие паттерны **изолируют знание о том, как, когда и какой именно объект создается**, делая клиентский код независимым от деталей инстанциирования.

---

2. Фабричный метод (Factory Method) и Абстрактная фабрика (Abstract Factory)

2.1. Фабричный метод (Factory Method)

Фабричный метод определяет общий интерфейс создания объекта, позволяя подклассам или фабричным функциям решать, какой конкретно класс инстанциировать.

```
# domain/factories.py
from abc import ABC, abstractmethod
from decimal import Decimal


class Ticket(ABC):
    @abstractmethod
    def get_payload(self) -> str: ...


class QrTicket(Ticket):
```

```
    def get_payload(self) -> str:
        return "BASE64_QR_DATA"


class NfcTokenTicket(Ticket):
    def get_payload(self) -> str:
        return "NFC_APDU_PAYLOAD"


class TicketIssuer(ABC):
    """Базовый эмитент: алгоритм общий, а метод создания — абстрактный."""

    @abstractmethod
    def create_ticket(self, fare: Decimal) -> Ticket:
        """Фабричный метод."""
        pass

    def issue_and_log(self, fare: Decimal) -> Ticket:
        # Общая бизнес-логика не зависит от типа билета!
        ticket = self.create_ticket(fare)
        print(f"[AUDIT] Выпущен билет: {ticket.get_payload()}")
        return ticket


class QrTicketIssuer(TicketIssuer):
    def create_ticket(self, fare: Decimal) -> Ticket:
        return QrTicket()


class NfcTicketIssuer(TicketIssuer):
    def create_ticket(self, fare: Decimal) -> Ticket:
        return NfcTokenTicket()
```

2.2. Абстрактная фабрика (Abstract Factory)

Предоставляет интерфейс для создания **семейств взаимосвязанных или взаимозависимых объектов**, не привязываясь к их конкретным классам.
Пример: Мультигородская транспортная экосистема. - Семейство «Алматы»: OnayCard, AlmatyValidator, AlmatyFareRule. - Семейство «Астана»: Transcard, AstanaValidator, AstanaFareRule.
Абстрактная фабрика гарантирует, что система никогда случайно не попытается валидировать астанинскую карту в несовместимом терминале Алматы.

---

3. Паттерн Строитель (Builder)

Строитель разделяет конструирование сложного объекта и его представление, позволяя пошагово собирать различные конфигурации объекта.
В системе Avtobys в конце смены формируется сложный финансово-аналитический отчет: - Период смены, номер маршрута, ID водителя; - Раздельные секции выручки (наличные, QR, транспортная карта, социальные льготники); - Список сбойных транзакций и метрики загруженности остановок.
Создавать такой отчет через конструктор с 15 параметрами (Report(a, b, c, d, e, f, g, ...)) — классический антипаттерн **Telescoping Constructor**.

Реализация Builder с Fluent Interface:

```python
# reports/builder.py
from dataclasses import dataclass, field
from decimal import Decimal
from typing import List


@dataclass
class DailyTransitReport:
    route_id: str = ""
    date: str = ""
    total_revenue: Decimal = Decimal("0.00")
    transactions_count: int = 0
    anomalies: List[str] = field(default_factory=list)


class TransitReportBuilder:
    def __init__(self):
        self._report = DailyTransitReport()

    def set_header(self, route_id: str, date: str) -> "TransitReportBuilder":
        self._report.route_id = route_id
        self._report.date = date
        return self

    def add_financials(self, revenue: Decimal, tx_count: int) -> "TransitReportBuilder":
        self._report.total_revenue = revenue
        self._report.transactions_count = tx_count
        return self

    def log_anomaly(self, message: str) -> "TransitReportBuilder":
        self._report.anomalies.append(message)
        return self

    def build(self) -> DailyTransitReport:
        # Валидация инвариантов отчета перед выдачей
        if not self._report.route_id:
            raise ValueError("Отчет не может быть сформирован без указания route_id.")
        return self._report
Использование в коде:
report = (
    TransitReportBuilder()
    .set_header(route_id="32", date="2026-10-06")
    .add_financials(revenue=Decimal("145000.00"), tx_count=1450)
    .log_anomaly("Сбой связи на остановке Саина-Шаляпина в 14:20")
    .build()
)
```

---

4. Почему Singleton — антипаттерн в современной разработке?

Паттерн **Singleton (Одиночка)** гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.

Почему в 2026 году Singleton считается антипаттерном?

1. **Глобальное разделяемое изменяемое состояние (Global Mutable State)**: делает поведение системы недетерминированным.
2. **Скрытые зависимости**: функции обращаются к синглтону напрямую (Config.get_instance().db_pass), вместо того чтобы явно объявить свою зависимость в конструкторе.
3. **Невозможность изоляции тестов**: тесты, запущенные параллельно, мутируют один и тот же синглтон и случайным образом валят друг друга (Flaky Tests).
4. **Нарушение SRP**: класс одновременно решает бизнес-задачу и управляет временем своей жизни в памяти.

Инженерная альтернатива:

Жизненным циклом одиночного объекта должен управлять **DI-контейнер** (container.register_singleton(Service)), а сам класс пишется как обычный класс без приватных конструкторов и глобальных статических переменных!

Контрольные вопросы и выводы

1. В каких случаях предпочтительнее использовать Builder вместо обычного конструктора с именованными аргументами?
2. Чем Abstract Factory отличается от Factory Method на концептуальном уровне?
3. Какие 4 проблемы порождает использование паттерна Singleton в модульном тестировании?
4. Как концепция внедрения зависимостей (DI) решает задачу единственности объекта без использования Singleton?

Литература к лекции

1. **Гамма Э. и др.** Паттерны проектирования. – СПб.: Питер, 2020. (Глава 3: Порождающие паттерны).
2. **Фаулер М.** Шаблоны корпоративных приложений. – М.: Вильямс, 2016.
3. **Refactoring.Guru**: Порождающие паттерны проектирования (https://refactoring.guru/ru/design-patterns/creational-patterns).